

HIERARCHICAL AI METEOROLOGIST

Verification Engine — Complete Technical Reference

Rule Engine · Causal (Causa) Engine · Data Fact Validator · Temporal Checker · Confidence Scorer

Code-level Documentation | June 2026

Source: [app/verification/](#) | [app/llm/meteorologist.py](#) | [app/schemas.py](#)

5 Verification Components Each serving a distinct validation role	11+ Active Rules in <code>met_rules.yaml</code> covering core weather physics	Dual-Path Firing Claims AND Causal Chain validation	Weighted Harmonic Score Zeroed on any critical failure
---	---	---	--

0. System Overview & Pipeline Flow

How all 5 verification engines are orchestrated inside `run_meteorologist()`

0.1 High-Level Architecture

The Hierarchical AI Meteorologist is a three-block pipeline: **Block 1** (Assistant) collects and structures weather data into a `ContextPayload`; **Block 2** (Meteorologist) calls the Gemini LLM and then runs **five parallel verification engines** against the LLM output; **Block 3** (Writer) formats the validated analysis into a final report. The PDF focuses entirely on the five verification engines within Block 2.

Stage	Component	File	Role
Block 1	Assistant Pipeline	<code>app/assistant/context_builder.py</code>	Fetches weather data → <code>ContextPayload</code>
Block 2 – LLM	Gemini Meteorologist	<code>app/llm/meteorologist.py</code>	Generates: summary, proof, claims[], causal_chain[]
Block 2 – V1	DataFactValidator	<code>app/verification/data_validator.py</code>	Fact-checks each claim against observed DataFrames
Block 2 – V2	MeteorologicalRuleEngine	<code>app/verification/rule_engine.py</code>	Checks claims & causal chain vs. physics rules (YAML)
Block 2 – V3	CausalGraph	<code>app/verification/causal_graph.py</code>	Validates causal_chain[] transitions via directed graph
Block 2 – V4	TemporalConsistencyChecker	<code>app/verification/temporal_checker.py</code>	Cross-scale directional coherence check
Block 2 – Score	ConfidenceScorer	<code>app/verification/confidence_scorer.py</code>	Combines V1–V4 into overall_score
Block 2 – Log	EvaluationTracker	<code>app/verification/evaluator.py</code>	Persists run metrics to <code>.cache/evaluation_log.json</code>
Block 3	Writer	<code>app/llm/writer.py</code>	Formats final FinalReport

0.2 Orchestration in `run_meteorologist()` [`meteorologist.py`, Lines 27–148]

All five engines are instantiated in `run_meteorologist()` (`app/llm/meteorologist.py`). After the Gemini call returns JSON, the engines are called sequentially:

```
# Lines 52-57 – Instantiation
fact_validator = DataFactValidator(thresholds_path) # → data_validator.py
rule_engine = MeteorologicalRuleEngine(met_rules_path) # → rule_engine.py
causal_graph = CausalGraph(met_rules_path) # → causal_graph.py
temporal_checker = TemporalConsistencyChecker() # → temporal_checker.py
scorer = ConfidenceScorer(weights_path) # → confidence_scorer.py

# Lines 80-89 – Sequential validation after LLM call
claims = data.get('claims', []) # List[WeatherClaim dicts]
causal_chain = data.get('causal_chain', []) # List[str] e.g. ['pressure_drop',
'wind_increase']

fact_results = fact_validator.validate_all_claims(claims, hourly_df, six_hour_df, daily_df,
current_df)
rule_results = rule_engine.validate_rules(claims, causal_chain)
causal_results = causal_graph.validate_chain(causal_chain)
temporal_results = temporal_checker.validate_temporal_consistency(claims)
report = scorer.compute_report(fact_results, rule_results, causal_results, temporal_results)
```

Config files

met_rules.yaml (rules), thresholds.yaml (numeric categories), verification_weights.yaml (component weights) — all under app/verification/

1. MeteorologicalRuleEngine (Rule Engine)

app/verification/rule_engine.py | Class: MeteorologicalRuleEngine | Lines 9–492

1.1 Purpose

The Rule Engine is an **if-then physics rule checker**. It answers: *"If the LLM's output claims condition X, does the output also include the physically expected consequence Y within an appropriate time lag?"* Rules are declared in **met_rules.yaml** (antecedent → consequent pairs). Violations are marked as **high-severity** by the Confidence Scorer.

1.2 Core Concepts

Concept	Explanation
Antecedent	The condition that triggers (fires) a rule. Can be a single dict or a list. Each antecedent has: variable, condition, threshold.
Consequent	The meteorological outcome expected after the antecedent fires. Contains: variable, condition, threshold.
Firing	A rule 'fires' when ALL its antecedents are found — either in the claims list (Check A) or in the causal chain (Check B).
Satisfied	A fired rule is 'satisfied' when the consequent is also found in the appropriate time window.
Penalty Weight	Each rule carries a confidence_weight. Violated rules contribute penalty_weight = confidence_weight to the score calculation.
Dual-Path	Check A: claims list. Check B: causal_chain nodes. If Check A fires but no consequent exists in the window, Check B is used.

1.3 Key Methods & Where They Live

`__init__()` [Lines 10–41]

Loads met_rules.yaml via `_load_rules()` and populates two critical dictionaries:

```
# node_synonyms [Lines 16–21] – Normalizes LLM variant node names
self.node_synonyms = {
    'humidity_rise': 'high_humidity',
    'frontal_passage': 'cooling',
    'clearing': 'no_rain',
    'fog_formation': 'low_visibility',
}

# event_to_var_dir [Lines 23–41] – Maps causal chain node names to (variable, direction)
# pairs
self.event_to_var_dir = {
    'pressure_drop': ('pressure_change_6h', 'down'),
    'wind_increase': ('wind_speed', 'up'),
    'high_humidity': ('relative_humidity', 'up'),
    'rainfall': ('precipitation', 'up'),
    'thunderstorm': ('weather_category', 'up'),
    # ... 11 entries total
}
```

`_get_rule_var_direction()` [Lines 55–85]

Translates a rule's YAML variable + condition string into a canonical (variable_type, direction) tuple. Direction is one of 'up', 'down', or 'neutral'.

```
# Direction mapping logic [Lines 60-65]
if cond in ['less_than', 'less_than_or_equal', 'decreasing', 'drop', 'cold', 'dry']:
    direction = 'down'
elif cond in ['greater_than', 'greater_than_or_equal', 'increasing', 'rise', ...]:
    direction = 'up'
else:
    direction = 'neutral'

# Variable normalization map [Lines 67-84]
var_map = {
    'pressure_change_6h': 'pressure_change_6h',
    'pressure_change_3h': 'pressure_change_6h', # alias
    'relative_humidity': 'relative_humidity',
    'temperature': 'temperature',
    'weather_category': 'weather_category',
    # ... 14 entries
}
```

_is_magnitude_matched() [Lines 167–230]

Checks whether a claim's threshold meets the rule's threshold with appropriate margin. Handles three threshold forms: **string/categorical**, **range list [min, max]**, and **single numeric**. Also applies a **50% guard** to prevent weak magnitude matches from firing a rule.

```
# 50% magnitude guard [Lines 214-221]
if cond in ['greater_than', ...]:
    if claim_val < 0.5 * rule_val: # Claim is less than 50% of rule threshold – discard
        return False
elif cond in ['less_than', ...]:
    if claim_val > 1.5 * rule_val: # Claim overshoots 150% – discard
        return False

# Variable-specific tolerances [Lines 123-141]
margins = {
    'temperature': 1.0, 'relative_humidity': 5.0, 'pressure': 2.0,
    'pressure_change_6h': 0.5, 'wind_speed': 1.0, 'precipitation': 0.5
}
```

validate_rules() [Lines 243–492] — The Main Entry Point

This is the primary public method. It iterates over every rule in met_rules.yaml and performs two independent checks:

Check	Mechanism	How it fires	Consequent verified by
A — Claims Path	<code>_claims_match_antecedent()</code> + time overlap	Allowed dependents found in claims[] with Searchapping built-in interval window [ant_start	Searchapping built-in interval window [ant_start
B — Causal Chain Path	<code>event_to_var_dir</code> lookup in normalized antecedent	Allowed antecedent events found in causal chain	Causal chain antecedent appears only after all antecedent inc

Fallback Logic (Lines 418–453):

```

fired = fired_via_claims OR fired_via_chain

IF fired_via_claims:
IF claims_consequent_satisfied: → satisfied = True (done)
ELIF NOT has_consequent_claims_in_window:
→ Fallback to chain result (chain acts as backup)
ELSE:
→ satisfied = False (claims present but contradicted, no chain fallback)

ELIF fired_via_chain:
→ Result depends solely on chain_consequent_satisfied

ELSE (not fired):
→ satisfied = True (rule irrelevant, no penalty)

```

Scoring (Lines 477–492):

```

fired_rules = [r for r in results if r['fired']]
total_weight = sum(r['weight'] for r in fired_rules)
satisfied_weight= sum(r['weight'] for r in fired_rules if r['consequent_satisfied'])
consistency_score = satisfied_weight / total_weight # 0.0 → 1.0
# If no rules fired → consistency_score = 1.0 (no violations)

```

2. CausalGraph (the 'Causa Engine')

[app/verification/causal_graph.py](#) | Class: `CausalGraph` | Lines 6–290

2.1 Purpose

The `CausalGraph` is a **graph-theoretic plausibility checker**. It answers: *"Is the LLM's `causal_chain`] a physically possible sequence of weather events, where each step can be reached from the previous via known meteorological pathways?"* It uses a **NetworkX directed graph (DiGraph)** built from `met_rules.yaml` rules plus hard-coded system links.

2.2 Key Difference: Rule Engine vs. Causa Engine

Dimension	MeteorologicalRuleEngine (Rule Engine)	CausalGraph (Causa Engine)
Input analyzed	<code>claims</code>] — structured LLM assertions with timestamps and thresholds	<code>causal_chain</code>] — list of weather event strings (e.g. ['pressure', 'temp', 'precip', 'wind'])
Structure	Procedural if-then rule evaluation in Python loops	Graph-theoretic path search on a NetworkX DiGraph
Threshold awareness	YES — checks numeric magnitudes and lag windows	NO — categorical/structural only (no numbers)
Time awareness	YES — validates UTC timestamp windows for claims	NO — only checks ordering/sequence in the list
Firing condition	All rule antecedents matched in <code>claims</code> OR chain	Each consecutive pair in the chain has a path in the graph
Forbidden edge check	NO explicit forbidden edges	YES — <code>self.invalid_edges</code> set blocks physically impossible transitions
Score output	<code>rule_consistency_score</code> ∈ [0.0, 1.0]	<code>validity_score</code> ∈ [0.0, 1.0]; 0.0 if any forbidden edge
Config weight	0.10 in <code>verification_weights.yaml</code>	0.20 in <code>verification_weights.yaml</code>
Role in overall score	Additive (excluded from harmonic mean zeroing)	Participates in harmonic mean; can zero entire score

2.3 Graph Construction — `__init__()` and `_load_rules_and_build_graph()` [Lines 7–188]

During initialization the `CausalGraph` builds the directed graph from two sources:

Source A: YAML Rules (Lines 121–167)

Every rule in `met_rules.yaml` is converted into one or more directed edges: **antecedent node** → **consequent node**. Edges carry confidence weight, lag hours, and `rule_id`.

```
for rule in rules: # Line 130
    target = self._map_var_to_node(...) # Consequent → node name
    sources = [self._map_var_to_node(ant)] # Antecedent(s) → node names
    for source in sources:
        self.graph.add_edge(
            normalized_source, normalized_target,
            confidence=confidence, lag=lag, rule_id=rule_id
        )
```

Source B: System Links (Lines 169–188)

14 hard-coded edges fill gaps not covered by explicit rules, ensuring connectivity for common weather patterns:

ID	Edge	Conf.	Meteorological Basis
SYS_LINK_001	low_visibility → fog_persistence	0.90	Fog visibility links to fog persistence
SYS_LINK_002	high_humidity → low_visibility	0.80	High humidity causes visibility reduction
SYS_LINK_003	stable_pressure → light_winds	0.80	High pressure suppresses wind
SYS_LINK_004	light_winds → no_rain	0.75	Calm conditions suppress precipitation
SYS_LINK_005	no_rain → warming	0.70	Clear skies allow daytime heating
SYS_LINK_006	warming → dry_air	0.75	Heating reduces relative humidity
SYS_LINK_007	dry_air → extreme_heat	0.65	Dry air amplifies heat extremes
SYS_LINK_008	extreme_heat → cooling	0.70	Evening radiational cooling follows heat
SYS_LINK_009	pressure_drop → high_humidity	0.80	Low pressure draws in moist air
SYS_LINK_010	wind_increase → thunderstorm	0.75	Wind shear promotes convection
SYS_LINK_011	fog_persistence → cooling	0.70	Fog blanket suppresses daytime warming
SYS_LINK_012	wind_gusts → warming	0.65	Foehn/downslope wind causes warming
SYS_LINK_013	rainfall → thunderstorm	0.80	Heavy rain indicates convective activity
SYS_LINK_014	thunderstorm → wind_gusts	0.85	Thunderstorm downdrafts create gusts

Forbidden Edges (Lines 24–30)

The **invalid_edges** set contains physically impossible transitions. If any of these appears in the causal chain, the **validity_score** is **forced to 0.0** regardless of other transitions:

```
self.invalid_edges = {
    ('dry_air', 'rainfall'), # Dry air cannot produce rain
    ('dry_air', 'fog_formation'), # No moisture → no fog
    ('dry_air', 'fog_persistence'), # Fog cannot persist without moisture
    ('cooling', 'extreme_heat'), # Cooling cannot lead to heat
    ('clearing', 'rainfall'), # Clear skies cannot produce rain
}
```

2.4 Node Normalization — `_normalize_node()` [Lines 34–78]

Because the LLM may use varied phrasing, this method converts free-text event strings to canonical graph node names using an **exact synonym lookup** followed by **keyword-based phrase matching**:

```
# Exact synonym lookup [Lines 39-40]
if s in self.node_synonyms: return self.node_synonyms[s]

# Keyword phrase matching [Lines 43-76]
if 'pressure_drop' in s or 'trough' in s: → 'pressure_drop'
if 'thunderstorm' in s or 'storm' in s: → 'thunderstorm'
if 'humidity' in s and 'drop' in s: → 'dry_air'
if 'humidity' in s: → 'high_humidity'
if 'fog' in s or 'visibility' in s: → 'low_visibility'
if 'wind' in s and 'light' in s: → 'light_winds'
if 'wind' in s: → 'wind_increase'
```

2.5 validate_chain() [Lines 190–289] — Main Entry Point

Iterates over every consecutive pair **(u, v)** in the normalized chain and applies 3 checks:

Step	Check	Result if passes	Result if fails
1	Is (u, v) in invalid_edges?	Continue to next check	has_forbidden_edges=True; score forced 0.0; append
2	Does graph.has_edge(u, v) exist?	Append edge.confidence to transition_score	Continue to Step 3
3	Is there any path u → v in graph?	Score = cumulative path confidence / path length	Append (0, any_end_in_chain) transitions

Missing Intermediates (Lines 266–273):

After per-pair checks, the method also identifies **expected intermediate events** that exist on the shortest path from chain[0] to chain[-1] but are absent from the chain. These are reported as low-severity omissions.

```
shortest_path = nx.shortest_path(self.graph, start_node, end_node)
for node in shortest_path[1:-1]:
    if node not in normalized_chain:
        missing_intermediates.append(node) # Low severity flag
```

3. DataFactValidator

app/verification/data_validator.py | Class: DataFactValidator | Lines 28–514

3.1 Purpose

The DataFactValidator answers: "Are the quantitative claims made by the LLM actually supported by the raw weather DataFrames?" It takes each **WeatherClaim** object (from the Meteorologist's output) and finds the matching rows in the hourly, six-hour, or daily DataFrames, then validates the claim's assertion type against those observed values.

3.2 WeatherClaim Schema [schemas.py, Lines 116–125]

```
class WeatherClaim(BaseModel): # schemas.py line 116
    claim_id: str # e.g. 'CLAIM_001'
    variable: str # e.g. 'temperature', 'wind_speed'
    assertion_type: str # 'increasing'/'decreasing'/'category_match'/'exceeds_threshold'...
    window_start_utc: str # ISO timestamp (start of claim window)
    window_end_utc: str # ISO timestamp (end of claim window)
    threshold_or_delta: str # e.g. '> 25.0', '+5.0', 'heavy', '1.32 to 3.32'
    data_scale: Literal['hourly', 'six_hour', 'daily', 'climatology', 'current']
```

3.3 validate_all_claims() [Lines 187–514] — Main Entry Point

Processes each claim through 4 sequential steps:

Step	Code Location	Description
1. Extract claim attrs	Lines 218–225	Safely reads claim fields supporting both dict and object formats. Resolves source_scale / data_scale.
2. Time-window filter	Lines 240–277	Selects DataFrame rows where timestamp falls within [window_start_utc, window_end_utc].
3. Column resolution	Lines 279–314	_get_column_name() maps variable + scale → actual DataFrame column. If non-numeric, routes to CASE C.
4. Assertion validation	Lines 331–512	Routes to one of 3 cases based on assertion_type.

3.4 Three Assertion Validation Cases

CASE A: Directional Trends [Lines 334–376]

Triggered when `assertion_type ∈ {'increasing', 'decreasing', 'trend'}`. Uses linear regression slope + monotonic step ratio:

```
slope = float(np.polyfit(np.arange(len(values)), values, 1)[0]) # linear regression
diffs = np.diff(values) # consecutive differences

# For 'increasing':
matching_steps = sum(diffs > 0) # count positive steps
confidence = matching_steps / total_steps # fraction of positive steps
is_pass = (slope > 0.001) AND (confidence >= 0.5) # must trend AND > 50% steps go up
```

CASE B: Category Match / Comparison to Normal [Lines 378–459]

Triggered when `assertion_type` ∈ `{'category_match', 'comparison_to_normal'}`. Looks up `thresholds.yaml` to find the numerical range for the category string (e.g. 'heavy' → precipitation [7.6, 50.0]):

```
threshold_cfg = self._lookup_threshold(variable, threshold_or_delta, assertion_type)
# Returns dict: {'min': 7.6, 'max': 50.0} OR {'val': -4.0} OR {}

# If min/max present → Range check
matching_count = sum((values >= min_val) & (values <= max_val))
is_pass = (matching_count / len(values)) >= 0.5 # 50% of window must be in range

# If 'val' present → Delta check (temperature drop/rise)
delta = mean(last_third) - mean(first_third) # robust delta across window
is_pass = delta <= expected_val # for 'drop'; delta >= expected_val for 'rise'
```

CASE C: Threshold / Numeric Condition [Lines 461–512]

Default case for assertion types like 'exceeds_threshold', 'less_than', or generic. Parses the operator from `threshold_or_delta` string:

```
# _parse_numeric_condition() [Lines 144–160]
# Handles: '> 25.0', '>= 8.0', '< 10', '== 5', '1.32 to 3.32' (range)

op, expected_val = _parse_numeric_condition(threshold_or_delta)

# For each operator:
matching_count = sum(values > expected_val) # for op '>'; similarly for >=, <, <=, ==
confidence = matching_count / len(values)
is_pass = confidence >= 0.25 # Pass if condition met in ≥25% of window
```

3.5 Column Name Mapping [Lines 46–93]

`_get_column_name()` maps user-friendly variable names to actual DataFrame column names based on scale:

Variable	hourly/current column	six_hour/daily column
temperature	t_c	t_mean_c
relative_humidity	rh_percent	rh_mean_percent
wind_speed	wind_speed_ms	wind_speed_mean_ms
wind_gust	wind_gust_ms	wind_speed_mean_ms
precipitation	precipitation_mm	precipitation_sum_mm
visibility	visibility_m	visibility_mean_m
pressure	pressure_hpa	pressure_mean_hpa

4. TemporalConsistencyChecker

app/verification/temporal_checker.py | Class: TemporalConsistencyChecker | Lines 6–141

4.1 Purpose

The Temporal Checker answers: "Does the LLM's narrative stay directionally consistent across different time scales?" It prevents contradictions like a daily claim of 'temperature increasing' while an overlapping hourly claim says 'temperature decreasing'. It operates purely on **directional logic** (up/down/neutral) — no numeric magnitudes.

4.2 Three Pairwise Comparisons [Lines 104–117]

Comparison	Code Line	Scale Pairs Checked
Daily vs 6-Hour	Lines 105–107	Each daily claim × each six_hour claim — overlap-checked
6-Hour vs Hourly	Lines 109–112	Each six_hour claim × each hourly/current claim — overlap-checked
Daily vs Hourly	Lines 114–117	Direct daily × hourly (Problem 3 — skips six_hour intermediary)

4.3 Contradiction Detection Logic [Lines 74–102]

```
def check_directional_contradiction(c1, c2, scale1, scale2):
    # 1. Skip if variables differ
    if _normalize_variable(c1.variable) != _normalize_variable(c2.variable): return

    # 2. Check temporal overlap
    if max(start1, start2) <= min(end1, end2): # Claims share a time window
        dir1 = _get_claim_direction(c1) # 'up'/'down'/'neutral'
        dir2 = _get_claim_direction(c2)

    # 3. Contradiction if both non-neutral and opposite directions
    if dir1 != 'neutral' AND dir2 != 'neutral' AND dir1 != dir2:
        contradictions.append({'severity': 'high', ...})
```

4.4 Direction Resolution [Lines 31–39]

_get_claim_direction() inspects both assertion_type and threshold_or_delta keywords:

```
if 'decreasing' in assertion OR 'drop'/'cold'/'dry'/'<' in threshold: → 'down'
if 'increasing' in assertion OR 'rise'/'heat'/'strong'/'>' in threshold: → 'up'
else: → 'neutral'
```

4.5 Scoring [Lines 119–133]

```
penalty = 0.0
for contra in contradictions:
    if contra['severity'] == 'high': penalty += 1.0
    elif contra['severity'] == 'medium': penalty += 0.5

consistency_score = max(0.0, 1.0 - (penalty / checks_count)) # scaled by total checks
is_consistent = not any(contra['severity'] == 'high' for contra in contradictions)
```

5. ConfidenceScorer

app/verification/confidence_scorer.py | Class: ConfidenceScorer | Lines 5–183

5.1 Purpose

The ConfidenceScorer is the **final aggregation layer**. It combines the four component scores into a single **overall_score** $\in [0.0, 1.0]$ using a **weighted harmonic mean** (for three components) plus an additive contribution from the rule engine. Critically: **if any of the three harmonic components is 0.0, the overall score is forced to 0.0**.

5.2 Weight Configuration [verification_weights.yaml]

Component	YAML Key	Default Weight	Role in Formula
DataFactValidator	data_fact	0.35	Harmonic mean (can zero total)
MeteorologicalRuleEngine	met_rule	0.10	Additive only (excluded from harmonic)
CausalGraph	causal_graph	0.20	Harmonic mean (can zero total)
TemporalConsistencyChecker	temporal_consistency	0.20	Harmonic mean (can zero total)

5.3 Score Computation [Lines 73–85]

```
non_rule_keys = ['data_fact', 'causal_graph', 'temporal_consistency']

# Critical zero-check – if any harmonic component is 0.0, overall = 0.0
if any(scores[k] == 0.0 for k in non_rule_keys):
    overall_score = 0.0
else:
    # Weighted harmonic mean of the 3 non-rule components
    w_sum = sum(weights[k] for k in non_rule_keys) # = 0.75
    w_denom= sum(weights[k] / scores[k] for k in non_rule_keys)
    base_harmonic = w_sum / w_denom # harmonic of 3 components

    # Rule engine added linearly on top
    w_rule = weights['met_rule'] # = 0.10
    overall_score = (base_harmonic * w_sum + scores['met_rule'] * w_rule) / (w_sum + w_rule)
```

5.4 Failure Classification [Lines 87–146]

Source	Severity	Trigger Condition	Message Format
DataFactValidator	medium	is_pass == False	Claim failed fact-checking: {reason} (Obs: {obs}, Exp: {exp})
MeteorologicalRuleEngine	high	fired AND NOT consequent_satisfied	Meteorological rule violation: {explanation}
CausalGraph (invalid)	high	invalid_transitions not empty	Causal chain invalid transition: {u} → {v} (Physically forbidden: {bo
CausalGraph (missing)	low	missing_intermediates not empty	Omitted expected intermediate event: '{node}'
TemporalConsistencyChecker	high/medium	contradictions list not empty	Cross-scale consistency clash on '{variable}': {reason}

5.5 Summary Text Generation [Lines 148–176]

```
if overall_score >= 0.8: → 'High meteorological confidence and physically coherent.'  
elif overall_score >= 0.5: → 'Moderate confidence with minor contradictions.'  
else: → 'Low confidence – critical meteorological discrepancies.'  
  
# Appends component scores and counts: X high-severity, Y medium, Z low
```

6. met_rules.yaml — Complete Rules Reference

app/verification/met_rules.yaml | Used by both Rule Engine AND CausalGraph

6.1 Rule File Structure

```
# Single-antecedent rule (antecedent: singular):
- rule_id: 'RULE_001'
  name: 'Baric Wind Rule'
  antecedent:
    variable: pressure_change_6h
    condition: less_than_or_equal
    threshold: -3.0
  consequent:
    variable: wind_speed
    condition: greater_than_or_equal
    threshold: 8.0
  confidence_weight: 0.75
  expected_lag_hours: 3.0

# Multi-antecedent rule (antecedents: list):
- rule_id: 'RULE_016'
  antecedents:
    - { variable: relative_humidity, condition: greater_than, threshold: 90.0 }
    - { variable: pressure_change_6h, condition: between, threshold: [-1.0, 1.0] }
    - { variable: wind_speed, condition: less_than_or_equal, threshold: 3.4 }
  consequent:
    variable: fog_persistence
    condition: equals
    threshold: 'persistent'
```

6.2 Active Rules Summary

Rule ID	Name	Antecedent → Consequent	Weight	Lag (h)
RULE_001	Baric Wind Rule	pressure_change_6h ≤ -3.0 → wind_speed ≥ 8.0	0.75	3.0
RULE_002	Saturated Air Precipitation	humidity > 90% → precipitation > 0	0.85	0.0
RULE_003	Thermal Low Development	temperature > 35°C → pressure_change < -1.5	0.60	2.0
RULE_004	Convective Instability Storm	weather_category = thunderstorm → precip ≥ 7.6	0.0	0.0
RULE_005	Cold Frontal Passage	temp_change_3h < -3°C → pressure_change_3h ≥ -8.0	0.5	1.0
RULE_006	Onshore Sea Breeze	temperature > 28°C → wind_speed > 4.0	0.70	1.0
RULE_007	Cyclone Severe Wind Risk	pressure_change_6h ≤ -6.0 → wind_speed ≥ 17.0	0.85	2.0
RULE_008	Dry Air Rain Suppression	humidity < 30% → precipitation = 0	0.95	0.0
RULE_009	Warm Frontal Advection	temp_change_6h > 3°C → pressure_change < -10.5	0.70	2.0
RULE_010	Diurnal Radiational Cooling	humidity < 40% → temp_change_6h < -4°C	0.70	6.0
RULE_011	Radiation Fog Formation	humidity ≥ 98% → visibility < 1000m	0.75	2.0

RULE_015	High Pressure Wind Suppression	pressure > 1025 hPa → wind_speed ≤ 3.4	0.80	0.0
RULE_016	Fog Persistence Rule	humidity>90% + stable_pressure + calm winds → fog_persistence	0.60	1.0
RULE_012–014	(Commented out)	Rainfall saturation, evaporative cooling, heavy rain-gusts	—	—

7. EvaluationTracker

app/verification/evaluator.py | Class: EvaluationTracker | Lines 22–242

7.1 Purpose

Persists run metrics to `.cache/evaluation_log.json` for trend analysis across pipeline runs. Called non-fatally in `meteorologist.py` (a failure does not crash the pipeline).

7.2 Metrics Computed [`_compute_metrics()`, Lines 34–131]

Metric	Formula	Meaning
false_claim_rate	failed_claims / total_claims	Fraction of WeatherClaims that failed DataFactValidator
cross_scale_consistent	NOT any(contradiction.severity == 'high')	Boolean: no high-severity temporal contradictions
verified_report	overall_score >= verification_threshold	Boolean: report passes confidence threshold
correction_needed	any failure.severity in ('high','medium')	Boolean: LLM output requires correction
hallucination_precision	corroborated_failures / failed_claims	Fraction of fact failures also confirmed by rule/causal failures
hallucination_recall	recalled_violations / total_rule_violations	Fraction of rule violations also caught by fact checking

How a single LLM response propagates through all 5 verification components

Hierarchical AI Meteorologist — Verification Engine Reference

9. Quick Reference — Classes, Methods & Files

Complete index of every significant method with file and line number

Class / Method	File	Lines	Returns
MeteorologicalRuleEngine.__init__()	rule_engine.py	10–41	Loads YAML, builds node_synonyms & event_to_var_dir
._load_rules()	rule_engine.py	43–49	Populates self.rules from met_rules.yaml
._normalize_node()	rule_engine.py	51–53	str → canonical node name
._get_rule_var_direction()	rule_engine.py	55–85	Tuple(var_type, direction)
._get_claim_var_direction()	rule_engine.py	87–121	Tuple(var_type, direction)
._get_variable_margin()	rule_engine.py	123–141	float tolerance for variable
._parse_claim_threshold()	rule_engine.py	143–165	Tuple(op, value, range)
._is_magnitude_matched()	rule_engine.py	167–230	bool — claim threshold vs rule threshold
._claims_match_antecedent()	rule_engine.py	232–241	List[claim dicts] that match antecedent
.validate_rules() ★	rule_engine.py	243–492	Dict{rule_results, rule_consistency_score, has_violations}
CausalGraph.__init__()	causal_graph.py	7–32	Loads YAML, builds DiGraph, sets invalid_edges
._normalize_node()	causal_graph.py	34–78	str → canonical graph node
._map_var_to_node()	causal_graph.py	80–119	YAML variable+cond+val → graph node
._load_rules_and_build_graph()	causal_graph.py	121–188	Builds DiGraph edges from YAML + SYS_LINKS
.validate_chain() ★	causal_graph.py	190–289	Dict{is_valid, invalid_transitions, missing_intermediates, validity_
DataFactValidator.__init__()	data_validator.py	29–32	Loads thresholds.yaml
._get_column_name()	data_validator.py	46–93	str — DataFrame column name for variable+scale
._lookup_threshold()	data_validator.py	95–142	Dict — numeric range/value from thresholds.yaml
._parse_numeric_condition()	data_validator.py	144–160	Tuple(op, value) or ('range', (min,max))
._validate_string_claim()	data_validator.py	162–185	Tuple(is_pass, confidence, reason)
.validate_all_claims() ★	data_validator.py	187–514	List[ValidationResult]
TemporalConsistencyChecker.__init__()	temporal_checker.py	7–9	Sets tolerances dict
._normalize_variable()	temporal_checker.py	11–29	str → canonical variable name
._get_claim_direction()	temporal_checker.py	31–39	'up'/'down'/'neutral'
.validate_temporal_consistency() ★	temporal_checker.py	41–141	Dict{is_consistent, consistency_score, contradictions, checks_c
ConfidenceScorer.__init__()	confidence_scorer.py	6–14	Sets default weights, loads YAML overrides
.compute_report() ★	confidence_scorer.py	28–183	Dict{overall_score, component_scores, failures, summary}
EvaluationTracker.record_run() ★	evaluator.py	165–200	Dict of metrics for this run + persists to log

.get_summary()	evaluator.py	202–242	Dict of aggregated metrics across all runs
run_meteorologist() ★	meteorologist.py	27–148	Tuple[MeteorologistOutput, meta_dict]

★ = primary public entry-point method. All methods in `app/verification/` are imported and called by `run_meteorologist()` in `app/llm/meteorologist.py` (Lines 27–148).